Asymptotic Algorithm Analysis

- The asymptotic analysis of an algorithm determines the running time in big-Oh (big O) notation
- To perform the asymptotic analysis
  - We find the worst-case number of primitive operations executed as a function of the input size
  - We express this function with big-Oh notation
- Example:
  - We determine that algorithm ***arrayMax*** executes at most $8n - 5$ primitive operations
  - We say that algorithm ***arrayMax*** "runs in $O(n)$ time"
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Big O Rules 1.

- If $f(n)$ is a polynomial of degree d, then $f(n)$ is $O(n^d)$, i.e.,
  1. Drop lower-order terms
  2. Drop constant factors
- Use the smallest possible class of functions
  - Say "$2n$ is $O(n)$" instead of "$2n$ is $O(n^2)$"
- Use the simplest expression of the class
  - Say "$3n + 5$ is $O(n)$" instead of "$3n + 5$ is $O(3n)$"

Big O Rules 2.

1. If $T_1(n) = O(f(n))$ and $T_2(n) = O(g(n))$ then
   (a) $T_1(n) + T_2(n) = O(\max(f(n), g(n)))$
   (b) $T_1(n) * T_2(n) = O(f(n) * g(n))$

2. If $T(n) = (\log n)^k$ then $T(n) = O(n)$

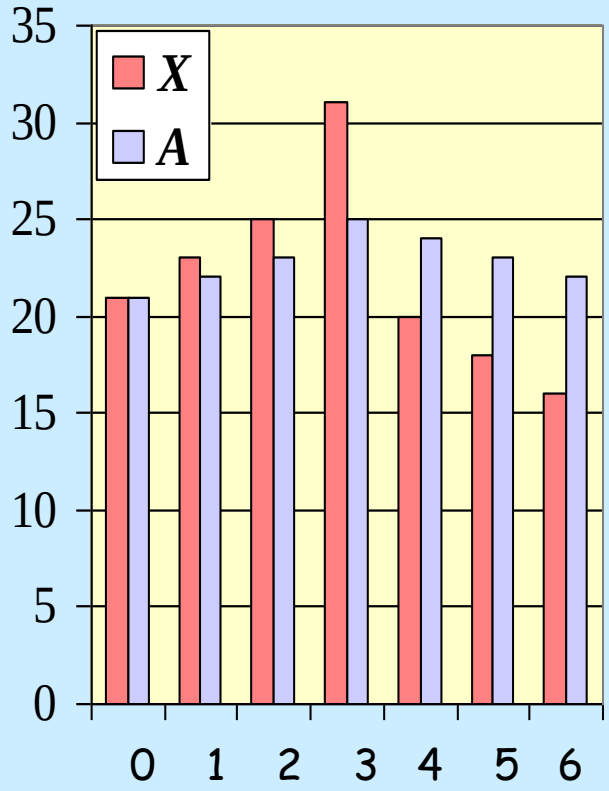
True for any value of k. So logs grow very slowly

Computing Prefix Averages

- We further illustrate asymptotic analysis with two algorithms for prefix averages
- The i-th prefix average of an array X is average of the first $(i + 1)$ elements of X:

$$A[i] = (X[0] + X[1] + \ldots + X[i])/(i+1)$$

- Computing the array A of prefix averages of another array X has applications to financial analysis



Prefix Averages (Quadratic)

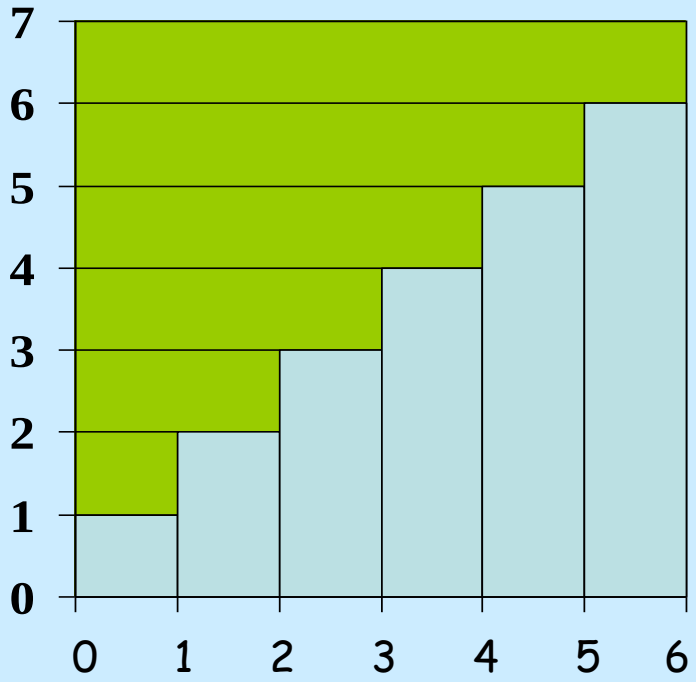
- The following algorithm computes prefix averages in quadratic time by applying the definition

Algorithm *prefixAverages1*(*X*, *n*)	
Input array *X* of *n* integers	
Output array *A* of prefix averages of *X*	#operations
$A \leftarrow$ new array of n integers	n
for $i \leftarrow 0$ **to** $n - 1$ **do**	n
$s \leftarrow X[0]$	n
for $j \leftarrow 1$ **to** i **do**	$1 + 2 + \ldots + (n - 1)$
$s \leftarrow s + X[j]$	$1 + 2 + \ldots + (n - 1)$
$A[i] \leftarrow s / (i + 1)$	n
return A	1

Note, this time ignoring constants

Aside: Arithmetic Progression

- The running time of ***prefixAverages1*** is $O(1 + 2 + \ldots + n)$
- The sum of the first n integers is $n(n + 1) / 2$
  - There is a simple visual proof of this fact



Thus, algorithm ***prefixAverages1*** runs in $O(n^2)$ time

Prefix Averages (Linear)

- The following algorithm computes prefix averages in linear time by keeping a running sum

Algorithm ***prefixAverages2***(***X***, ***n***)	
Input array ***X*** of ***n*** integers	
Output array ***A*** of prefix averages of ***X***	#operations
A ← new array of ***n*** integers	***n***
s ← 0	1
for ***i*** ← 0 **to** ***n*** – 1 **do**	***n***
s ← ***s*** + ***X***[***i***]	***n***
A[***i***] ← ***s*** / (***i*** + 1)	***n***
return ***A***	1

- Algorithm ***prefixAverages2*** runs in $O(n)$ time – so more efficient

Some more growth rates

most common

- $O(1)$ or *constant*
- $O(\log n)$ or *logarithmic* (to base 2)
- $O(n)$ or *linear*
- $O(n \log n)$ (usually just called n log n)
- $O(n^2)$ or *quadratic*

Exercise for you:

- How fast does $\sqrt{n}$ grow? Where would you place it in the following table? (Hint, assume that N is an even power of 2 …)

A little note about functions.. See board.

n	1	log n	n log n	n^2
1	1	0	0	1
2	1	1	2	4
4	1	2	8	16
8	1	3	24	64
16	1	4	64	256
32	1	5	160	1024
64	1	6	384	4096
128	1	7	896	16384
256	1	8	2048	65536
512	1	9	4608	262144
1024	1	10	10240	1048576
2048	1	11	22528	4194304
4096	1	12	49152	16777216
8192	1	13	106496	67108864
16384	1	14	229376	268435456

General rules for finding running times

- Rule 1 – Loops
  - The running time of a loop is at most the running time of the statements inside the loop (including tests) multiplied by the number of iterations
- Rule 2 – nested Loops
  - Should be analysed inside out. Total running time of a statement inside a group of nested loops is running time of statement multiplied by the product of the sizes of all the loops.

```
Algorithm Test1(n):
    for i=1 to n do
        for j=1 to n do
            for k=1 to n do
                increment x
```

.. is $O(n^3)$

General rules contd.

- Rule 3 – Consecutive statements
  - These just add (so take the maximum O, see slide 4)

- Rule 4 – If-then-else
  - Running time is never more than the time of the test (condition) plus the larger of the running times of S1 and S2

```
Algorithm Test2(n):
    if condition1 then
        S1
    else
        S2
```

Similarly for multiple/nested else statements

Examples

```
Algorithm Squares1(n):
    Input: Integer n
    Output: The sum of the first n positive integers
    i←0
    sum←0
    while (i<n)
      increment i
      sum=sum+i*i
    return sum
```

Is $O(n)$

- With some mathematical knowledge we can rewrite the program so that it is faster: $\sum i^2 = n*(n+1)*(2n+1)/6$

```
Algorithm Squares2(n):
    Input: Ingeger n
    Output: The sum of the first n positive integers
    sum←(n*(n+1)*(2*n+1))/6
    return sum
```

Is $O(1)$

```
Algorithm IntSqrt1(n):
    Input: Ingeger n
    Output: The integer part of the square root of n
    i←0
    while ((i+1)*(i+1)<n) do
      increment i
    return i
```

Is $O(\sqrt{n})$

- Faster version: "hone in" on $\sqrt{n}$

```
Algorithm IntSqrt2(n):
    Input: Ingeger n
    Output: The integer part of the square root of n
    L ←0
    H ←n
    D ←0
    while H ≠ L+1 do
       D= (L+H)/2
       if (D*D<=n) then
           L=D
       else
         H=D
    return L
```

Is $O(\log n)$

Complexity of the simple search and sorting algorithms

In COS101 you saw that:

- Search
  - LinearSearch – n/2 "comparisons"
  - BinarySearch – $\log_2 n$ "comparisons"

- Sorting
  - InsertionSort, SelectionSort: about $n^2/2$ "comparisons"

Actually BubbleSort also $n^2/2$ (see later this lecture!)

More formal analysis (like we're doing here) would give:

LinearSearch is $O(n)$

Binary search cost is $O(\log_2 n)$,

InsertionSort is $O(n^2)$ etc.

Bubble sort is discussed in examples class 1

(More) Definitions

- If T(n) is a function of n then
  - $T(n) = O(f(n))$ if there are constants c and n_0 such that $T(n) \leq cf(n)$ when $n \geq n_0$
  - $T(n) = \Omega(g(n))$ if there are constants c and n_0 such that $T(n) \geq cg(n)$ when $n \geq n_0$
  - $T(n) = \Theta(h(n))$ if and only if $T(n) = O(h(n))$ and $T(n) = \Omega(h(n))$
  - $T(n) = o(p(n))$ if $T(n) = O(p(n))$ and $T(n) \neq \Theta(h(n))$

big omega

big theta

little o

Don't panic! Will really only use the first of these, and we will translate them into English first…

In English?

- $T(n) = O(f(n))$
  - Can we find a constant c such for large enough n, $T(n) \leq cf(n)$?
- $T(n) = \Omega(g(n))$
  - Can we find a constant c such that for large enough n, $T(n) \geq cg(n)$?
- $T(n) = \Theta(h(n))$
  - Is the bound as tight as we can get?
- $T(n) = o(p(n))$
  - Could we perhaps have done better?

Previous examples…

Squares1 Time= $A+Bn = O(n)$ (actually $\Theta(n)$)
Squares2 Time= $C = O(1)$ (actually $\Theta(1)$)
IntSqrt1 Time = $A+B\sqrt{n} = O(\sqrt{n})$ (actually $\Theta(\sqrt{n})$)
IntSqrt2 Time = $A + B \log_2 n = O(\log_2 n)$ (actually $\Theta(\log_2 n)$)

- Why are they all Θ instead of just O?
- Won't use $\Theta(n)$ any more!

Because we didn't have to overestimate

Note:

- can go from function to O notation, but not vice versa
- often O is easier to find than the function itself.

A note about logarithms in running times

IntSqrt2 was $O(\log_2 n)$ because every time we iterated loop, H-L was divided by 2. (Similarly, binary search).

Consider following example:

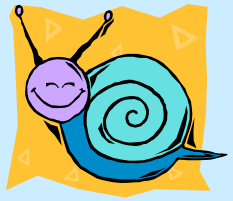
```
while n >=1 do
   n → n/3
```

Do we say that this fragment is $O(\log_3 n)$?

No! we say it is $O(\log_2 n)$ - why?

Traditionally, only use logs to the base 2

Two other complexities (both impossibly slow)



- Exponential – $O(2^n)$
  - E.g. print out all of the *combinations/subsets* from a set of size n.
  - (Every element is either in or out of a subset)
- Factorial – $O(n!)$
  - E.g. print out all of the *permutations* of the n elements of a set. Could use recursion here.

A handy lookup table

Function	O	Description	How good?
A	$O(1)$	constant	nearly immediate
$A+B\log_2 n$	$O(\log n)$	logarithmic	stupendously fast
$A+B\sqrt{n}$	$O(\sqrt{n})$	square root	very fast
$A+Bn$	$O(n)$	linear	fast
$A+B\log_2 n+Cn$	$O(n)$	linear	fast
$A+Bn\log_2 n$	$O(n \log n)$	n log n	fairly fast
$A+Bn+Cn\log_2 n$	$O(n \log n)$	n log n	fairly fast
$A+Bn+Cn^2$	$O(n^2)$	quadratic	slow for large n
$A+Bn+Cn^2+Dn^3$	$O(n^3)$	cubic	slow for most n
$A+B2^n$	$O(2^n)$	exponential	impossibly slow
$An!$	$O(n!)$	factorial	impossibly slow

Best, average and worst cases

- The analysis so far depends only on the size of the problem and not on the disposition of data.
- In linear search we could be
  - lucky and find the item immediately - complexity $O(1)$
  - unlucky and find the item last - complexity $O(n)$

  otherwise we have average case here - complexity $O(n)$

- A full complexity analysis should include
  - best case
  - worst case
  - average case

Note, in some cases (like most of our examples) these would all be equal

Space Complexity

- We can analyse algorithms for space complexity.
- Space complexity is less important than time complexity - we can buy memory but not time.
- Note that linked lists are more efficient space-wise than arrays. We return unused memory back to storage pool.

Time Complexity and ADTs

- We will see (chapter 5) that we usually have a choice of representation in an ADT.
- We should be guided by the complexity of the most often performed task.

A historical note:

" Big Omicron and big Omega and big Theta" by Donald Knuth, in *ACM SIGACT News. Volume 8, Issue 2, April – June 1976*

A bit of fun:

"The complexity of songs", by Donald Knuth, Communications of the ACM, April 1984. pp.18 - 24

See ADS moodle webpage.

Some examples for you to try..

1

```
sum←0
for i=1 to N do
  for j=1 to i do
     sum=sum+1
```

2

```
sum←0
for i=1 to N do
  for j=1 to i*i do
     for k=1 to j do
        sum=sum+1
```

3

```
sum←0
for i=1 to N do
  for j=1 to i*i do
    if j mod i = 0 then
     for k=1 to j do
        sum=sum+1
```

In all cases give an analysis of running time (big O will do.. but make it as small as possible)

A note about logs

- If $x = b^c$ then $\log_b x = c$

  So $\log_{10} 1000 = 3$ and $\log_2 1024 = 10$

- For complexity analysis we generally use log x to mean $\log_2 x$

- The main rules we use about logs are:

  - $\log(xy) = \log x + \log y$ and
  - $\log x^y = y \log x$

So $\log_2 1024 = \log_2 (32*32) = \log_2 32 + \log_2 32 = 5 + 5 = 10$

Need to revise logs? Look in GoTa p. 154-155. Will do some examples in examples class.

Bubble sort is also discussed in example sheet 1

Bubble sort

Simple but inefficient.

To sort the following list into ascending order

3 2 1 6 4 5

First stage

Select the first item, 3. Is it bigger than the next, 2? Yes - swap

2 3 1 6 4 5

Select the second item, 3. Is it bigger than the next, 1? Yes- swap.

Repeat until the biggest is rightmost.

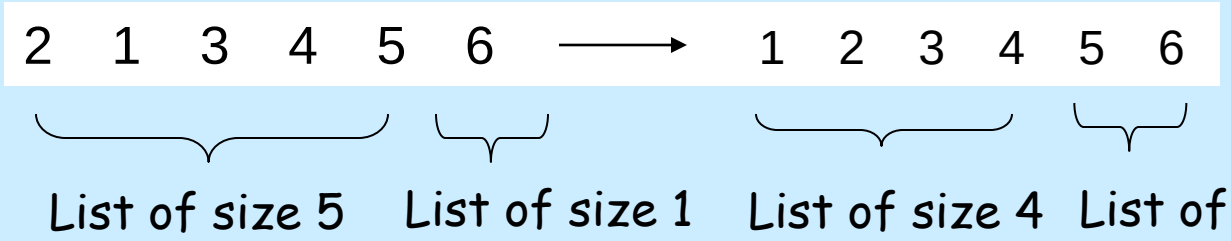
3	2	1	6	4	5	original
2	3	1	6	4	5	3,2 compared
2	1	3	6	4	5	3,1 compared
2	1	3	6	4	5	3,6 compared
2	1	3	4	6	5	6,4 compared
2	1	3	4	5	6	6,5 compared

5 comparisons in total

List of 6 items split into list of one item (the biggest) to the right and a list of 5 items (not necessarily in their original order) to the left.

Second Stage

Take the leftmost list of 5 items and do the same thing.



No change in 5th place, but takes
4 comparisons
- You check

Complexity

For N items in list:
Number of comparisons at first stage is N-1
Number of comparisons at second stage is N-2
...
Total=(N-1)+...+2+1=(N-1)N/2≈N²/2 for large N
But could stop when zero swaps made...

This is as bad as insertion sort and selection sort!

	Insertion_Sort	Selection_Sort
Minimum	$1 + 1 + \ldots + 1 = n-1$	$n-1 + n-2 + \ldots + 1 = n(n-1)/2$
Maximum	$1 + 2 + \ldots + n-1 = n(n-1)/2$	$n-1 + n-2 + \ldots + 1 = n(n-1)/2$
Average	$(1 + 2 + \ldots + n-1)/2 = n(n-1)/4$	$n-1 + n-2 + \ldots + 1 = n(n-1)/2$